

UCIe 3.0 TX Logical PHY

Verification Environment Specification

UCIe Graduation Project

May 7, 2026

Contents

1	1. Overview	4
2	2. UVM Architecture	4
2.1	DUT Overview	4
2.2	UVM Architecture	4
3	3. Packages	5
3.1	3.1 tx_defs_pkg	6
3.2	3.2 tx_tb_pkg	6
4	4. Interfaces	6
4.1	4.1 RDI Interface (rdi_if)	7
4.2	4.2 LTSM Interface (ltsm_if)	7
4.3	4.3 TX2Link Interface (tx2link_if)	7
5	5. Agents	8
5.1	5.1 RDI Agent (Active)	8
5.2	5.2 LTSM Agent (Active)	8
5.3	5.3 TX2Link Agent (Passive)	8
5.4	5.4 Reactive Stimulus Flow	9
6	6. Environment	9
6.1	6.1 tx_env_cfg	9
6.2	6.2 tx_env	9
7	7. Sequence Items	10
7.1	7.1 rdi_seq_item	10
7.2	7.2 ltsm_seq_item	10
7.3	7.3 tx2link_item	10
8	8. Sequences	11
8.1	8.1 ltsm_base_seq	11
8.2	8.2 rdi_base_seq	11
8.3	8.3 tx_virtual_seq	11
8.4	8.4 sqr_pool	11
9	9. Functional Coverage	11
9.1	9.1 LTSM Covergroup (cg_ltsm)	11
9.2	9.2 RDI Covergroup (cg_rdi)	11
10	10. Assertions (SVA)	11
10.1	10.1 Checking Strategy	12
10.2	10.2 Tri-State Assertions	12
10.3	10.3 LTSM Status Assertions	12
10.4	10.4 Per-State tx_done Timing	12
10.5	10.5 Cover Properties	12
11	11. Reference Model & Scoreboard	13
11.1	11.1 Predictor (tx_predictor)	13
11.1.1	11.1.1 TX Datapath Pipeline	13
11.2	11.2 Scoreboard (tx_scoreboard)	15

12	12. Configuration & Plusargs	16
13	13. Testbench Top (tx_tb_top)	16
14	14. File Organization	16

1 1. Overview

This document describes the UVM-based verification environment for the **TX Logical PHY** of a UCIe 3.0 (Universal Chiplet Interconnect Express) die-to-die link. The TX path is responsible for accepting flit data from the D2D Adapter via the RDI (Raw Die-to-Die Interface), processing it through a datapath pipeline (byte-to-lane mapping, LFSR scrambling, lane reversal), and driving serialized output on 16 physical lanes toward the analog front-end.

The environment verifies the TX Controller logic by driving the full LTSM (Link Training State Machine) encoding sequence — spanning initialization, training, and active data transfer — while injecting flit traffic on the RDI and observing the serialized egress output. A **reactive stimulus** architecture coordinates the three interface agents through TLM analysis FIFOs, ensuring protocol-correct temporal ordering without hard-coded synchronization.

2 2. UVM Architecture

2.1 DUT Overview

The figure below shows the top-level testbench module (`tx_tb_top`) and its connectivity to the DUT. Three SystemVerilog interfaces — `rdi_if`, `ltsm_if`, and `tx2link_if` — form the boundary between the verification environment and the TX Logical PHY. The SVA assertion module is bound directly to the DUT outputs.

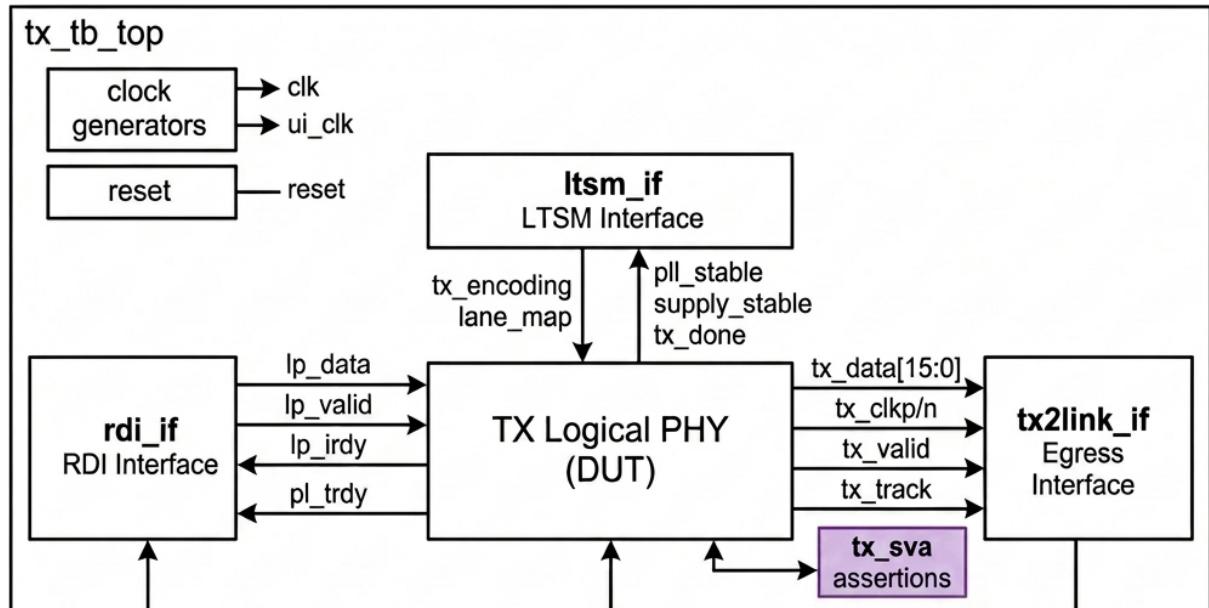


Figure 1: DUT Overview

2.2 UVM Architecture

The figure below illustrates the UVM environment (`tx_env`) hierarchy, showing the three agents, TLM analysis connections, and the data flow through the predictor, scoreboard, and coverage collector.

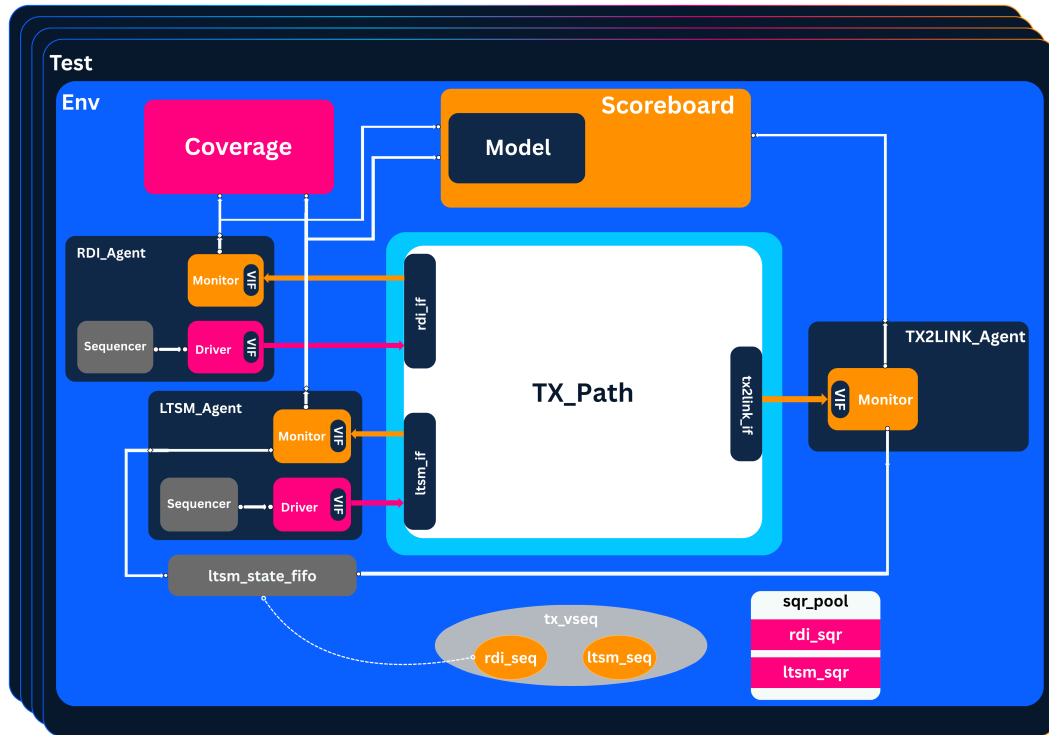


Figure 2: UVM Architecture

The environment follows a standard UVM layered architecture with the following key components:

Component	Role
RDI Agent (Active)	Drives flit data into the DUT; reacts to back-pressure
LTSM Agent (Active)	Drives LTSM state encodings to control the DUT FSM
TX2Link Agent (Passive)	Monitors serialized egress output from the DUT
Model	Reference model — generates expected egress output
Scoreboard	Compares golden predictions against actual observations
Coverage	Collects functional coverage on states, transitions, and lane maps
SVA	SVA module — protocol assertions on DUT-driven signals

3 3. Packages

Two SystemVerilog packages centralize all definitions and file includes.

3.1 3.1 tx_defs_pkg

The definitions package provides all shared types, parameters, and helper functions used across the environment.

Parameters:

- `NUM_LANES` = 16 — physical lane count.
- `DEFAULT_NBYTES` = 256 — default flit size in bytes.

Enumerations:

- `flit_size_e` — supported flit sizes: 64, 128, 256, 512 bytes.
- `ltsm_state_group_e` — TB-level constraint groups: `HAPPY_PATH`, `PATTERN_GEN`, `TRAIN_ERROR`, `TRISTATE`, `ACTIVE`.
- `ltsm_encoding_e` — full 9-bit LTSM FSM encoding covering all states across four sub-FSMs (Initialization, Training, Active, Data-to-Clock Test).

Lane Map Constants:

- `LANE_MAP_DEGRADE_NOT_POSSIBLE` (3'b000), `LANE_MAP_LANES_0_TO_7` (3'b001), `LANE_MAP_LANES_8_TO_15` (3'b010), `LANE_MAP_ALL_FUNCTIONAL` (3'b011).

Helper Functions:

Function	Description
<code>get_fsm_id()</code>	Extracts the 2-bit <code>FSM_ID</code> from a 9-bit encoding
<code>is_init_fsm()</code>	Returns 1 if encoding is in FSM 00 (Initialization)
<code>is_train_fsm()</code>	Returns 1 if encoding is in FSM 01 (Training)
<code>is_active_fsm()</code>	Returns 1 if encoding is in FSM 10 (Active)
<code>is_d2c_fsm()</code>	Returns 1 if encoding is in FSM 11 (Data-to-Clock Test)
<code>is_tristate_state()</code>	Returns 1 for states requiring Hi-Z output (RESET, SBINIT, PARAM, CAL)
<code>is_active_data()</code>	Returns 1 only for the <code>ACTIVE</code> encoding
<code>is_pattern_gen_state()</code>	Returns 1 for pattern generation states
<code>uses_lane_map()</code>	Returns 1 for REPAIRMB/REPAIR states where <code>lane_map</code> is relevant

3.2 3.2 tx_tb_pkg

The testbench package imports UVM and `tx_defs_pkg`, then includes all verification component files in strict dependency order (sequence items → configuration → agents → sequences → reference model → environment → tests). This is the single import point for the entire TB.

4 4. Interfaces

Three SystemVerilog interfaces model the DUT boundaries. Each provides dedicated `modport` declarations for driver, monitor, and DUT roles.

4.1 RDI Interface (`rdi_if`)

The RDI interface carries flit payloads between the D2D Adapter and the TX Logical PHY using ready/valid handshaking.

Signal	Width	Direction (TB → DUT)	Description
<code>lp_data</code>	[NBYTES-1:0] [7:0]	Input	Flit payload as a 2D byte array
<code>lp_valid</code>	1-bit	Input	Asserted when <code>lp_data</code> carries a valid flit
<code>lp_irdy</code>	1-bit	Input	Adapter intent-to-send
<code>pl_trdy</code>	1-bit	Output	PHY backpressure — when low, adapter must hold signals stable

Parameterized by NBYTES (default 256). Clocked on the logical clock domain (`clk`).

4.2 LTSM Interface (`ltsm_if`)

The LTSM interface bypasses the internal state machine, acting as a direct control vector into the TX Controller.

Signal	Width	Direction (TB → DUT)	Description
<code>tx_encoding</code>	9-bit	Input	Active LTSM state/substate encoding
<code>lane_map</code>	3-bit	Input	Width degradation configuration
<code>pll_stable</code>	1-bit	Output	PLL lock indicator
<code>supply_stable</code>	1-bit	Output	Power supply rails stable
<code>tx_done</code>	1-bit	Output	Current operation complete

Clocked on the logical clock domain (`clk`).

4.3 TX2Link Interface (`tx2link_if`)

The TX-to-Link interface is the final physical output boundary before the analog serial drivers.

Signal	Width	Direction	Description
<code>tx_data</code>	16-bit	Output	16 serial data lanes (1 bit/UI each)
<code>tx_clkp</code> / <code>tx_clkn</code>	1-bit each	Output	Forwarded differential clock pair
<code>tx_valid</code>	1-bit	Output	Physical valid lane
<code>tx_track</code>	1-bit	Output	Training synchronization signal

Operates on the fast UI clock domain (`ui_clk`). Half-rate: 1 fast clock = 2 UI. During disabled/reset states, all signals are expected to be Hi-Z.

5. Agents

The environment contains **three agents** — two active and one passive — each targeting a distinct DUT interface.

5.1 RDI Agent (Active)

Architecture: Driver + Monitor + Sequencer.

- **Driver (`rdi_driver`):** Implements a **reactive backpressure** model. It drives `lp_data`, `lp_valid`, and `lp_irdy`, then monitors `pl_trdy` at each clock edge. While `pl_trdy` is de-asserted, the driver holds all outputs stable — no data is lost. Once `pl_trdy` re-asserts, the transfer completes and the next flit is requested from the sequencer.
- **Monitor (`rdi_monitor`):** Passively detects completed flit transfers by sampling when `lp_valid && lp_irdy && pl_trdy` are all high simultaneously. Extracts `active_flit_size` bytes from the interface and broadcasts a `rdi_seq_item` via its analysis port.
- **Sequencer:** Standard UVM sequencer parameterized on `rdi_seq_item`.

Note

Note: The RDI agent does **not** check `pl_trdy` correctness — this is a DUT output. The driver only reacts to it for protocol compliance.

5.2 LTSM Agent (Active)

Architecture: Driver + Monitor + Sequencer.

- **Driver (`ltsm_driver`):** Drives `tx_encoding` and `lane_map`, then applies **state-dependent wait rules**:
- RESET → blocks until `pll_stable && supply_stable` (DUT signals).
- Pattern generation / apply states → blocks until `tx_done` (DUT signal).
- All other states → holds for a randomized delay period.
- **Monitor (`ltsm_monitor`):** Detects encoding transitions by comparing the current `tx_encoding` against the previous value each clock cycle. On change, it broadcasts a `ltsm_seq_item` containing the new encoding and `lane_map`. This monitor is the **central hub for cross-agent reactivity** — both the RDI sequence and the TX2Link monitor subscribe to its analysis port.
- **Sequencer:** Standard UVM sequencer parameterized on `ltsm_seq_item`.

Note

Note: The LTSM agent does **not** verify `pll_stable`, `supply_stable`, or `tx_done` — these are DUT outputs checked by the SVA module.

5.3 TX2Link Agent (Passive)

Architecture: Monitor only (no driver or sequencer).

- **Monitor (tx2link_monitor):** A state-aware assembler that operates on the fast `ui_clk` domain. It receives LTSM state updates via a **TLM analysis FIFO** connected to the LTSM monitor, and uses the current state to determine chunk sizes (number of UI cycles to sample). The monitor is "blind" — it samples all 16 data lanes for the chunk duration regardless of state, tagging each chunk with `captured_state` metadata. The scoreboard interprets which lanes carry valid data.
- Active/Pattern states: 64 fast-clock samples per chunk.
- Handshake-only states: no output expected (skip).

Each write to the analysis port contains $64 \text{ UI samples} \times 16 \text{ lanes} = 128 \text{ bytes}$ of lane data, reducing write overhead compared to per-UI transactions.

5.4 Reactive Stimulus Flow

Cross-agent coordination uses **TLM analysis FIFOs** rather than hard-coded synchronization:

LTSM Monitor	ap	ltsm_to_rdi_fifo	RDI
Sequence (gates flit gen on ACTIVE)			
determines chunk sizes)		tx2link_monitor.ltsm_state_fifo (	
		tx_predictor (updates internal state)	
		tx_coverage (samples LTSM covergroups)	

The RDI sequence blocks on the FIFO until the LTSM enters **ACTIVE**, then generates flits. If the state leaves **ACTIVE**, flit generation halts and the sequence re-waits. This decouples stimulus timing from agent internals.

6 Environment

6.1 tx_env_cfg

Centralized configuration object distributed via `uvm_config_db`:

Field	Type	Description
<code>rdi_vif</code>	<code>virtual rdi_if</code>	RDI interface handle
<code>ltsm_vif</code>	<code>virtual ltsm_if</code>	LTSM interface handle
<code>tx2link_vif</code>	<code>virtual tx2link_if</code>	TX2Link interface handle
<code>rdi_agent_mode</code>	<code>uvm_active_passive_enum</code>	RDI agent mode (default:UVM_ACTIVE)
<code>ltsm_agent_mode</code>	<code>uvm_active_passive_enum</code>	LTSM agent mode (default:UVM_ACTIVE)

6.2 tx_env

The top-level environment instantiates all sub-components and wires TLM connections in the connect phase:

1. **LTSM Monitor** → **ltsm_to_rdi_fifo** — cross-agent reactivity for RDI sequence.
2. **LTSM Monitor** → **tx2link_monitor.ltsm_state_fifo** — state-aware chunk assembly.
3. **LTSM Monitor** → **Predictor** — state tracking for golden model.
4. **LTSM Monitor** → **Coverage** — LTSM covergroup sampling.

5. **RDI Monitor** → **Predictor** — flit data for datapath prediction.
6. **RDI Monitor** → **Coverage** — RDI covergroup sampling.
7. **Predictor** → **Scoreboard** (`golden_fifo`) — expected egress transactions.
8. **TX2Link Monitor** → **Scoreboard** (`actual_fifo`) — observed egress transactions.

Active agent sequencers are registered in the global `sqr_pool` (sequencer container pattern) for access by virtual sequences without requiring a virtual sequencer component.

7. Sequence Items

7.1 7.1 rdi_seq_item

Represents a single flit transaction on the RDI interface.

Field	Type	Description
<code>data</code>	<code>rand logic [7:0] []</code>	Dynamic byte array sized to <code>active_flit_size</code>
<code>delay</code>	<code>rand int unsigned</code>	Inter-flit gap in clock cycles (0–20)
<code>active_flit_size</code>	<code>static flit_size_e</code>	Per-run flit size (set via <code>+FLIT_SIZE plusarg</code>)

7.2 7.2 ltsm_seq_item

Represents a single LTSM state command.

Field	Type	Description
<code>encoding</code>	<code>rand ltsm_encoding_e</code>	Target 9-bit LTSM encoding
<code>lane_map</code>	<code>rand logic [2:0]</code>	Width degradation config (constrained to valid values)
<code>delay</code>	<code>rand int unsigned</code>	Inter-state delay in clocks (1–20)
<code>active_group</code>	<code>ltsm_state_group_e</code>	Selects which constraint group is active

Includes five mutually-exclusive constraint groups and static ordered state queues per group. The `set_group()` method enables exactly one constraint at a time.

7.3 7.3 tx2link_item

Represents an assembled chunk of serial data from the egress monitor. **Never randomized** — constructed exclusively by the monitor.

Field	Type	Description
<code>captured_state</code>	<code>ltsm_encoding_e</code>	LTSM state active during assembly
<code>ui_count</code>	<code>int unsigned</code>	Number of UI cycles in this chunk
<code>data_lanes</code>	<code>logic [15:0] []</code>	Sampled data per UI (16 lanes per entry)

Implements `do_compare()` with 4-state (`===`) comparison to properly handle Hi-Z values.

8 Sequences

8.1 8.1 ltсм_base_seq

Drives LSTM state transitions in either **sequential walk** (ordered state list) or **random** (constrained random within group) mode. Control knobs:

- **group** — selects the state group (GROUP_HAPPY_PATH by default).
- **is_random** — 0 for ordered walk, 1 for random pick.
- **num_iterations** — overridden by +ITER plusarg.

8.2 8.2 rdi_base_seq

A **reactive sequence** that blocks on a TLM FIFO until the LSTM state becomes **ACTIVE**, then generates randomized flits. If the state leaves **ACTIVE**, generation halts and the sequence re-waits. The number of flits per active window is controlled by the +ITER plusarg.

8.3 8.3 tx_virtual_seq

Orchestrates both sub-sequences in parallel using **fork/join**. Retrieves sequencer handles from the global **sqr_pool** — no virtual sequencer component is needed.

8.4 8.4 sqr_pool

A singleton sequencer container (Cummings/Glasser DVCon pattern) mapping string names to sequencer handles via an associative array. Eliminates the need for virtual sequencer hierarchies.

9 Functional Coverage

The **tx_coverage** component subscribes to both LSTM and RDI monitor analysis ports.

9.1 9.1 LSTM Covergroup (cg_lstm)

Coverpoint / Cross	Description
cp_encoding	All 9-bit LSTM encodings with bins grouped by phase (Init, Train, Active, D2C)
cp_lane_map	All four lane map configurations
cx_encoding_x_lane_map	Cross coverage of encoding × lane_map, with ignore bins for non-degradation states
cp_transitions	Key state transitions: any→TRAINERROR, TRAINERROR→RESET, LINKINIT→ACTIVE, ACTIVE→L1

9.2 9.2 RDI Covergroup (cg_rdi)

Currently a placeholder — to be populated with flit size, backpressure, and data pattern coverage bins.

10 Assertions (SVA)

The **tx_sva** module contains SystemVerilog Assertions that check **only DUT-driven output signals**. The active agents (RDI and LSTM) are intentionally kept free of output-checking logic

for the signals driven by the DUT.

10.1 10.1 Checking Strategy

Category	What is Checked	Where
DUT outputs from LTSM (pll_stable, supply_stable, tx_done)	Checked in (SVA) — not in the LTSM agent	
DUT output from RDI (pl_trdy)	Not checked — the RDI driver only reacts to it	
Egress signals (tx_clkp/n, tx_valid, tx_track)	Checked in (SVA) — offloaded from the passive monitor	
Egress data (tx_data[15:0])	Checked in the (SVA) when IDLE or Hi-Z + (Scoreboard)	

10.2 10.2 Tri-State Assertions

During RESET, SBINIT, PARAM, and CAL states, all egress outputs must be Hi-Z:

- `assert_data_tristate` — `tx_data === 16'hzzzz`
- `assert_valid_tristate` — `tx_valid === 1'bz`
- `assert_track_tristate` — `tx_track === 1'bz`
- `assert_clk_tristate` — `tx_clkp === 1'bz && tx_clkn === 1'bz`

This offloads idle/tristate checking from the passive agent to hardware assertions.

10.3 10.3 LTSM Status Assertions

- `assert_pll_stable` — `pll_stable` must assert 1 cycle after entering RESET.
- `assert_supply_stable` — `supply_stable` must assert 1 cycle after entering RESET.
- `assert_tx_done_valid_state` — `tx_done` may only rise during operation states (pattern gen + apply).

10.4 10.4 Per-State tx_done Timing

Assertion	State	Expected Latency
<code>assert_tx_done_repairclk</code>	REPAIRCLK_CLK_PATTERN_GEN	$128 \times 24 = 3072$ cycles
<code>assert_tx_done_repairval</code>	REPAIRVAL_VALID_PATTERN_GEN	$28 \times 8 = 1024$ cycles
<code>assert_tx_done_reversal_id</code>	REVERSAL_PER_LANE_ID_GEN	$128 \times 16 = 2048$ cycles
<code>assert_tx_done_d2c_tx</code>	D2C_TX_PATTERN_GEN	$128 \times 8 = 1024$ cycles
<code>assert_tx_done_d2c_rx</code>	D2C_RX_PATTERN_GEN	4000 cycles
<code>assert_tx_done_reversal_apply</code>	REVERSAL_APPLY	1 cycle
<code>assert_tx_done_repairmb</code>	REPAIRMB_APPLY_DEGRADE_HND	1 cycle
<code>assert_tx_done_repair</code>	REPAIR_APPLY_DEGRADE_HND	1 cycle

10.5 10.5 Cover Properties

- `cover_backpressure` — RDI backpressure event during ACTIVE state.

- `cover_flit_transfer` — successful flit transfer during **ACTIVE** state.

11 11. Reference Model & Scoreboard

11.1 11.1 Predictor (`tx_predictor`)

The predictor subscribes to both the RDI and LTSM monitors. It maintains internal copies of the current LTSM state and lane map, and produces golden `tx2link_item` transactions by modeling the TX datapath pipeline.

11.1.1 TX Datapath Pipeline

The figure below illustrates the datapath blocks inside the predictor and how they interconnect. The TX Controller decodes the current LTSM encoding and asserts enable signals to each downstream block, selecting the active data path through the output multiplexer.

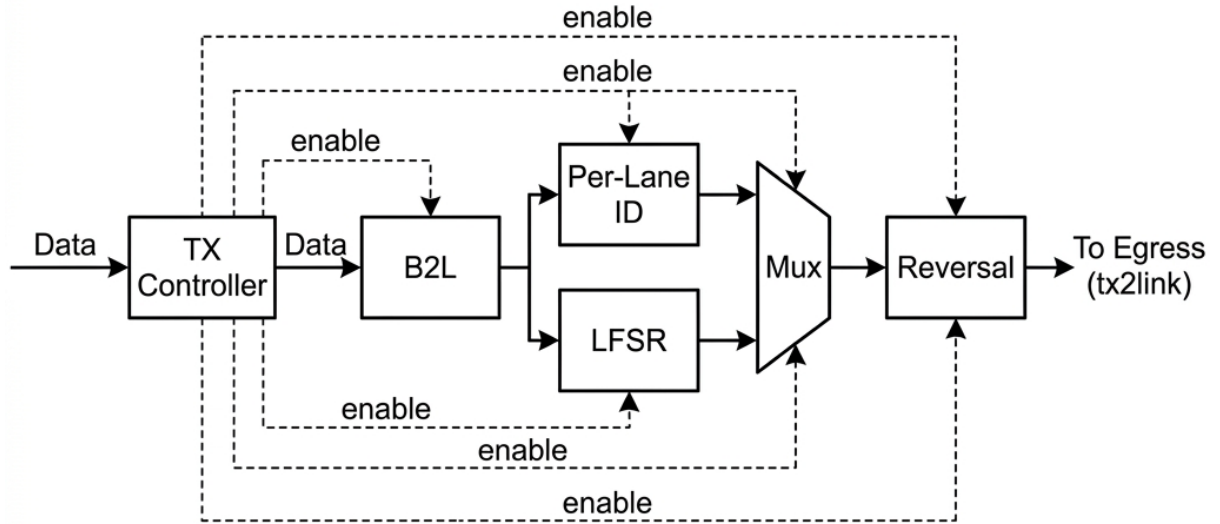


Figure 3: TX Datapath Pipeline

Block Descriptions:

TX Controller

- **Central Coordinator:** Acts as the primary router for the output path, dynamically enabling downstream reference models based on the current LTSM state.
- **Decoding Logic:** Utilizes state-based logic to assert targeted enable signals across the modeling environment. For example, during **ACTIVE** state it enables B2L + LFSR (scrambler mode), during **REVERSAL_PER_LANE_ID_GEN** it enables B2L + Per-Lane ID, and during **D2C_TX_PATTERN_GEN** it enables LFSR (pattern generator mode).

B2L (Byte-to-Lane Mapping)

- **Byte-to-Lane Mapping:** Distributes the incoming data stream across 16 parallel lanes, allocating exactly 4 bytes per lane.
- **Lane Degradation Support:** Implements protocol-compliant data striping for full x16 operation, seamlessly re-routing data payloads when the link degrades to x8 modes (utilizing either Lanes 0–7 or Lanes 8–15).
- **Protocol Compliance:** Implements the exact lane-mapping tables defined in the protocol specification for data striping.
- **Flow Control & Buffering:** Utilizes a FIFO queue to safely buffer incoming data transactions while the current transmission cycle completes.

LFSR

- **Dual-Mode Operation:** Functions as a standalone pattern generator during Link Training states, and switches to a data scrambler during the Active state.
- **Parallel Architecture:** Utilizes a fully unrolled polynomial implementation to calculate and output the next 64 bits of the sequence in a single clock cycle.

Per-Lane ID Generator

- **Unique Pattern Synthesis:** Generates protocol-compliant, unique identifier sequences (IDs) for each individual physical lane. Used during the REVERSAL_PER_LANE_ID_GEN state for lane identification and reversal detection.

Reversal

- **Lane Order Reversal:** Optionally reverses the physical lane ordering (Lane 0 ↔ Lane 15, Lane 1 ↔ Lane 14, etc.) when the reversal result from the REVERSAL state indicates a reversed connection to the link partner.
- **Enable Control:** The TX Controller asserts the reversal enable based on the outcome of the reversal detection phase. When disabled, lanes pass through unchanged.

Output Mux

- **Path Selection:** A 2:1 multiplexer controlled by the TX Controller that selects between the Per-Lane ID output (training/reversal detection) and the LFSR output (pattern generation or scrambled active data). The selected output feeds into the Reversal block before reaching the egress interface.

Datapath pipeline functions:	Stage	Function
	1. Byte-to-Lane Map	byte_to_lane_map()
	2. LFSR Scramble / Pattern Gen	lfsr_scramble()
	3. Per-Lane ID Generation	per_lane_id_gen()
	4. Lane Reversal	lane_reversal()
	5. Output Mux Select	output_mux_select()

11.2 11.2 Scoreboard (tx_scoreboard)

Receives golden items from the predictor and actual items from the TX2Link monitor via two independent TLM analysis FIFOs. Comparison is **event-based** — both FIFOs block until data is available.

Checks performed:

1. **State metadata match** — `captured_state` must agree.
2. **UI count match** — chunk sizes must be equal.
3. **Data comparison** — field-level `compare()` using 4-state logic for Hi-Z correctness.

Reports match/mismatch statistics in `report_phase`.

12. Configuration & Plusargs

Plusarg	Parsed By	Default	Description
+FLIT_SIZE=N	rdi_seq_item	256	Flit size in bytes (64, 128, 256, 512)
+ITER=N	rdi_base_seq, ltsm_base_seq	10	Number of iterations (flits or state transitions)
+UVM_TESTNAME=name	UVM core	—	Selects the test class to run

The `tx_env_cfg.parse_plusargs()` method is reserved for future test knobs.

13. Testbench Top (tx_tb_top)

The top module instantiates:

- **Clocks:** `clk` at 100 MHz (logical domain), `ui_clk` at 1 GHz (fast UI domain).
- **Reset:** Active-low, de-asserts after 100 ns.
- **Interfaces:** `rdi_if`, `ltsm_if`, `tx2link_if` — all published to `uvm_config_db`.
- **DUT Stub:** Minimal connectivity (always-ready `p1_trdy`, delayed `p1l_stable`/`supply_stable`, pulsed `tx_done`, Hi-Z egress). To be replaced with the actual RTL.
- **SVA Bind:** Commented out; to be enabled upon RTL integration.

14. File Organization

```

tb/
+-- packages/
|   +-- tx_defs_pkg.sv           # Enums, parameters, helper functions
|   +-- tx_tb_pkg.sv            # Single-import testbench package
+-- interfaces/
|   +-- rdi_if.sv               # RDI interface (parameterized)
|   +-- ltsm_if.sv              # LTSM controller interface
|   +-- tx2link_if.sv           # Egress physical interface
+-- agents/
|   +-- rdi_agent/              # Active: driver, monitor, sequencer
|   +-- ltsm_agent/             # Active: driver, monitor, sequencer
|   +-- tx2link_agent/          # Passive: monitor only
+-- seq_items/
|   +-- rdi_seq_item.sv          # Flit transaction
|   +-- ltsm_seq_item.sv         # LTSM state command
|   +-- tx2link_item.sv         # Assembled egress chunk
+-- seq_lib/

```



```
|   +-- sqr_pool.sv           # Sequencer container (singleton)
|   +-- rdi_base_seq.sv      # Reactive RDI sequence
|   +-- ltsm_base_seq.sv     # LTSM state walk sequence
|   +-- tx_virtual_seq.sv    # Top-level virtual sequence
+-- env/
|   +-- tx_env_cfg.sv        # Centralized configuration
|   +-- tx_env.sv           # Top-level UVM environment
+-- ref_model/
|   +-- tx_predictor.sv      # Golden reference model
+-- scoreboard/
|   +-- tx_scoreboard.sv     # Golden vs. actual comparison
+-- coverage/
|   +-- tx_coverage.sv       # Functional coverage collector
+-- assertions/
|   +-- tx_sva.sv           # SVA protocol assertions
+-- tests/
|   +-- tx_base_test.sv      # Base test (config, setup)
|   +-- tx_smoke_test.sv     # End-to-end smoke test
+-- top/
|   +-- tx_tb_top.sv         # Top module (clocks, reset, DUT stub)
```